Ders 10 Genişletilmiş Notu: Merge Sort ve Quick Sort: Rekürans Bağıntıları, Hoare Bölümlemesi ve Master Teoremi ile Analiz

Ders 10 Genişletilmiş Notu: Merge Sort ve Quick Sort: Rekürans Bağıntıları, Hoare Bölümlemesi ve Master Teoremi ile Analiz

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı
- 2. Merge Sort'un Çalışma Mantığı
- 3. Merge Sort Sözde Kodu
- 4. Merge Sort'un Rekürans Bağıntısı
- 5. Merge İşleminin En Kötü ve En İyi Durumu
- 6. Master Teoremi ile Çözüm
- 7. Quick Sort'un Çalışma Mantığı ve Bir Bölümleme İzi
- 8. Quick Sort ve Hoare Bölümlemesi Sözde Kodu
- 9. Quick Sort'un En İyi Durum Analizi
- 10. Gelecek Haftanın Planı
- Örnek Sorular

1. Dersin Amacı

Bu kayıt, haftanın uygulama saatidir. Hoca bu hafta uygulama yapılmayacağını, konu anlatımına devam edileceğini söyledi. Çünkü böl ve yönet (divide and conquer) konusunda iki algoritma daha kalmıştı: birleştirmeli sıralama (merge sort) ve hızlı sıralama (quick sort). Bu derste ikisinin matematiksel analizi yapıldı.

Hoca bu iki algoritmanın ayrıntısına çok girmediğini, çünkü ikisinin de daha önce Veri Yapıları dersinde anlatıldığını belirtti. Bu dersteki hedef farklıdır.

> **Hocanın vurgusu:** Bu dersin amacı rekürans bağıntısını (recurrence relation) yazabilmek ve yazdıktan sonra algoritmanın verimliliğini, büyüme derecesini (order of growth) hesaplayabilmektir. Amaç size merge sort'u öğretmek değildir.

2. Merge Sort'un Çalışma Mantığı

Hoca önce merge sort'un $n \log n$ karmaşıklıkta olduğunu hatırlattı ve sınıfa algoritmanın adımlarını sordu. Bir öğrenci şöyle özetledi: Dizi tek eleman kalana kadar bölünür. Sonra parçalar geri birleştirilirken elemanlar karşılaştırılır ve küçükten büyüğe yerleştirilir. İki eleman bir araya geldiğinde sıralı bir ikili oluşur. Sonra bu ikililer birleşip sıralı dörtlüler oluşturur ve bu şekilde yukarı doğru çıkılır. Yani her adımda bir birleştirme (merge) işlemi yapılır.

Hoca bu açıklamadan şu sonucu çıkardı: Merge sort'ta bölerken aslında bir iş yapılmaz, dizi sadece ortadan ikiye ayrılır. Asıl iş birleştirirken yapılır.

Hocanın vurgusu: Quick sort'ta bu durum tam tersidir. Orada asıl iş bölme sırasında yapılır, birleştirme görece basittir.

3. Merge Sort Sözde Kodu

3.1 Ana Fonksiyon

Hoca özyinelemeli (recursive) çözümün sözde kodunu (pseudocode) tahtaya yazdı. Elimizde 0'dan $n-1$'e kadar indislenen n elemanlı bir A dizisi vardır. Durma koşulu $n = 1$'dir. Tek elemanlı bir dizi zaten sıralıdır. Bu yüzden $n > 1$ olduğu sürece dizinin ilk yarısı yeni bir B dizisine, ikinci yarısı ($n/2$'den $n-1$'e kadar) yeni bir C dizisine kopyalanır. Kopyalama bittikten sonra algoritma kendini iki kez çağırır: önce B ile, sonra C ile. Hoca son satırı sınıfa sordu. Bir öğrenci "merge etmemiz gerekiyor" dedi. Son adımda B ve C birleştirilip sonuç A'ya yazılır.

```
Mergesort(A[0..n-1])
  if n > 1
    copy A[0..floor(n/2)-1] to B[0..floor(n/2)-1]
    copy A[floor(n/2)..n-1] to C[0..ceil(n/2)-1]
    Mergesort(B[0..floor(n/2)-1])
    Mergesort(C[0..ceil(n/2)-1])
    Merge(B, C, A)
```

3.2 Merge Fonksiyonu

Merge fonksiyonuna üç parametre gelir: $B[0..p-1]$, $C[0..q-1]$ ve $A[0..p+q-1]$. Hoca bu kısmın, iki sıralı diziyi birleştirme işi olarak, Veri Yapıları derslerinden zaten bilindiğini söyledi.

Önce dizi sınırlarının geçilip geçilmediği kontrol edilir: `while i < p and j < q`. Döngünün içinde iki taraftaki elemanlar karşılaştırılır. $B[i] \leq C[j]$ ise $B[i]$, $A[k]$'ya atanır ve i bir artırılır. Değilse $C[j]$, $A[k]$'ya atanır ve j bir artırılır. Her iki durumda da sonunda k bir artırılır. Döngüden çıkınca hangi dizi tamamlanmadıysa onun kalanı kopyalanır. $i = p$ ise B bitmiş, C tamamlanmamış demektir. Bu durumda $C[j..q-1]$, $A[k..p+q-1]$'e kopyalanır. Aksi halde $B[i..p-1]$ kopyalanır.

```
Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])
  i <- 0; j <- 0; k <- 0
  while i < p and j < q do
    if B[i] <= C[j]
      A[k] <- B[i]; i <- i + 1
    else
      A[k] <- C[j]; j <- j + 1
    k <- k + 1
  if i = p
    copy C[j..q-1] to A[k..p+q-1]
  else
    copy B[i..p-1] to A[k..p+q-1]
```

4. Merge Sort'un Rekürans Bağıntısı

4.1 Temel İşlem

Hoca rekürans bağıntısını sınıfa yazdırmak istedi. Bir öğrenci "master teoremine göre mi yapacağız?" diye sordu. Hoca, önce bağıntının yazılması gerektiğini, sonra çözüm için master teoreminin ya da geriye doğru yerine koymanın (backward substitution) kullanılmasının fark etmediğini söyledi.

İlk soru temel işlemin (basic operation) ne olduğuydu. Bir öğrenci kopyalama ya da atama dedi. Hoca, merge sort'un genel algoritmasına bakmak gerektiğini hatırlattı. Başka bir öğrenci "merge işlemi" dedi. Sonunda bir öğrenci Merge içindeki `if` karşılaştırmasını gösterdi. Hoca bunu onayladı: Burada her seferinde bir karşılaştırma (comparison) yapılır. Bu yüzden bağıntı $C(n)$ olarak, karşılaştırma sayısı üzerinden yazılır.

4.2 Bağıntının Parçaları

Bir öğrenci j için ayrı bir terim yazılıp yazılmayacağını sordu. Hoca şu an buna gerek olmadığını söyledi. Ardından bir öğrenci $2C(n/2)$ yazılacağını önerdi. Hoca bu terimi şöyle açıkladı: Problem ikiye bölünerek ele alınır, $n/2$ buna karşılık gelir. Baştaki 2 ise iki grubun da çözülmeye devam edildiğini gösterir. İkili arama (binary search) olsaydı C'nin başındaki katsayı 1 olurdu.

Sonra $f(n)$ yerine ne geleceği soruldu. Bir öğrenci "$n \log n$ mi?" dedi. Hoca büyüme derecesini sormadığını, birleştirmek için hangi işin yapıldığını sorduğunu söyledi. Cevap $C_{merge}(n)$'dir. Çünkü birleştirme yapan bir fonksiyon vardır ve bu fonksiyon n tane girdi alır. Bağıntı $n > 1$ için geçerlidir.

Başlangıç koşulu (initial condition) için hoca "hangi noktada duracağım?" diye sordu. Tek eleman kalınca durulur ve artık karşılaştırma yapmaya gerek yoktur. Bu yüzden $C(1) = 0$ olur.

$$C(n) = 2C(n/2) + C_{merge}(n), \quad n > 1, \qquad C(1) = 0$$

4.3 En Kötü Durum İçin Yazmak

Hoca bağıntıyı genel olarak yazdıktan sonra en kötü durum (worst case) için hesapladığını söyledi. İkiye bölmede bir en kötü durum yoktur. En kötü durum birleştirme tarafında söz konusudur. Bu yüzden bağıntı şöyle yazılır:

$$C_w(n) = 2C_w(n/2) + C_{merge}(n)$$

Geriye $C_{merge}(n)$'nin en kötü durumdaki değeri kalır.

5. Merge İşleminin En Kötü ve En İyi Durumu

5.1 En Kötü Durum: $n - 1$ Karşılaştırma

Bir öğrenci, `while` döngüsüne p ve q'ya kadar girildiğini düşündüğünü, en kötü durumda döngünün tamamının dönüp dönmeyeceğini sordu. Hoca bunun dizinin bütünü üzerinden düşünülmesi gerektiğini söyledi: Dizinin bütününde n eleman vardır ve bunlar ikiye bölünüp birbiriyle karşılaştırılır.

Hoca önce $8, 3, 2, 9, 7, 1, 5, 4$ dizisini ikiye böldü. Birleştirme aşamasında iki yarı sıralı olacağı için yarılar $2, 3, 8, 9$ ve $1, 4, 5, 7$ olur. En kötü durumda her iki gruptan sırayla birer eleman alınır. Bu da birleştirmenin en kötü durumda $n - 1$ karşılaştırmadan geçmesi demektir.

Hoca sonra örneği silip daha açık bir örnek verdi: $2, 4, 6, 8$ ile $1, 3, 5, 7$ birleştiriliyor.

- 1. karşılaştırma: 1 ile 2. 1 yerleşir.
- 2. karşılaştırma: 2 ile 3. 2 yerleşir.
- 3. karşılaştırma: 3 ile 4. 3 yerleşir.
- 4. karşılaştırma: 4 ile 5. 4 yerleşir.
- 5. karşılaştırma: 5 ile 6. 5 yerleşir.
- 6. karşılaştırma: 6 ile 7. 6 yerleşir.
- 7. karşılaştırma: 7 ile 8. 7 yerleşir.

Sonunda B dizisinde yalnız 8 kalır ve karşılaştırmasız kopyalanır. Toplam $7 = n - 1$ karşılaştırma yapılır ($n = 8$). Elemanlar iki taraftan dönüşümlü geldiği için her adımda iki taraf da karşılaştırılmak zorundadır.

> **Ders sonrası not:** Hocanın ilk örneği olan $2, 3, 8, 9$ ile $1, 4, 5, 7$ birleştirilirken python ile sayıldığında 6 karşılaştırma yapılır, 7 değil. Çünkü 2 ve 3 art arda soldan gelir. Hoca da bu yüzden örneği değiştirip $2, 4, 6, 8$ ile $1, 3, 5, 7$'yi kullandı. Bu ikinci örnekte $7 = n - 1$ karşılaştırma doğrulandı.

Daha sonra bir öğrenci $f(n)$'in nasıl bulunduğunu anlamadığını söyledi. Hoca soruyu sınıfa yöneltti. Bir öğrenci şöyle açıkladı: Temel işlem olarak `if` karşılaştırması alındı. En kötü durumda, sonuncusu hariç bütün değerler sürekli birbiriyle karşılaştırılmak zorunda kalır. $n/2$ değer karşı taraftaki $n/2$ değerle karşılaştırılır ve `while` sürekli döner. "Eksi bir" olmasının sebebi, dizilerden birinin sonuna gelindiğinde döngüden çıkılıp kalanın kopyalanmasıdır. Hoca bunu onayladı.

5.2 En İyi Durum: $n/2$ Karşılaştırma

Hoca birleştirmenin bir de iyi durum senaryosu olduğunu gösterdi. Bir tarafta $1, 2, 3, 4$, öbür tarafta bundan büyük $5, 10, 100$ gibi sayılar olsun. Önce $1, 2, 3, 4$ yerleştirilir. Bundan sonra 5 ve sonrası otomatik olarak sonuç dizisine kopyalanır. Bu durumda $n-1$ karşılaştırmaya gerek yoktur. Elemanların dizilimine bağlı olarak $n/2$ karşılaştırma yapılıp gerisi kopyalanabilir.

> **Ders sonrası not:** Hoca ikinci grubu "1, 5, 10, 100" diye söyledi. Anlatılan durum için bu grubun bütün elemanları $1, 2, 3, 4$'ten büyük olmalıdır (örneğin $5, 10, 100, \ldots$). Python ile $[1, 2, 3, 4]$ ve $[5, 10, 100, 200]$ birleştirildiğinde $4 = n/2$ karşılaştırma çıktı. İkinci grupta gerçekten 1 olsaydı 5 karşılaştırma yapılırdı.

5.3 Merge İşleminin En Kötü Durumu ile Sıralamanın En Kötü Durumu

Soru: Normalde bir sıralamada dizinin en kötü hali tersten sıralı olmasıdır. Bu merge için geçerli değil mi?

Cevap: Hoca soruyu güzel bulduğunu söyledi. Burada en kötü durumdan bahsederken birleştirme işleminin (merge operation) en kötü durumundan bahsediliyor, merge sort'un sıralama kısmının en kötü durumundan değil. $n-1$ adım, birleştirme yapılırken oluşabilecek en kötü adım sayısıdır. Hoca, bir sıralamanın en kötü durum senaryosunun genellikle dizinin sıralanmak istenen yönün tersine sıralı verilmesi olduğunu, bunun quick sort'ta da konuşulacağını ekledi.

> **Hocanın vurgusu:** Merge sort'un en kötü durumu ile merge işleminin en kötü durumu farklı şeylerdir. Burada hesaplanan, merge işleminin en kötü durumudur.

Böylece en kötü durum bağıntısı tamamlanır:

$$C_w(n) = 2C_w(n/2) + n - 1, \quad n > 1, \qquad C_w(1) = 0$$

6. Master Teoremi ile Çözüm

Hoca bağıntının, isteğe göre geriye doğru yerine koymayla ya da bir böl ve yönet problemi olduğu için master teoremiyle çözülebileceğini söyledi. Master teoremi $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ biçimindeki bağıntılara uygulanır. Hoca parametreleri sınıfa sordu:

- $a = 2$: İki alt problem de çözülmeye devam edilir.
- $b = 2$: Dizi ikiye bölünür.
- $d = 1$: Bir öğrencinin açıklamasıyla, son kısım $n - 1$'dir ve n'in üssü 1'dir.

Hoca d seçimini ayrıca açıkladı: Daha önceki bir örnekte $f(n)$ sabit bir sayıydı (1), o zaman n^0 deyip $d = 0$ alınmıştı. Şimdi $f(n) = n - 1$'dir. -1 kısmı düşer, en büyük dereceli terim alınır. Bu yüzden $d = 1$'dir.

Sonra koşul soruldu. Bir öğrenci $a = b^d$ olduğunu söyledi: $2 = 2^1$. Bu durumda formül $\Theta(n^d \log n)$ verir. $d = 1$ olduğu için:

$$C_w(n) \in \Theta(n \log n)$$

Böylece merge sort'un büyüme derecesi, rekürans bağıntısı yazılarak bulunmuş oldu. Hoca bunun geriye doğru yerine koymayla da çözülebileceğini unutmamak gerektiğini, bunun önemli bir nokta olduğunu söyledi.

> **Ders sonrası not:** Kesin değer derste hesaplanmadı. $n = 2^k$ için bağıntının çözümü $C_w(n) = n\log_2 n - n + 1$'dir. Python ile $n = 2, 4, 8, 16, 32$ için bağıntı $1, 5, 17, 49, 129$ verdi ve formülle aynı çıktı. $n = 8$ için bütün permütasyonlar denendi. Merge sort'un en fazla karşılaştırma sayısı da 17 çıktı.

Hoca ayrıca merge sort'un teorik olarak teorik minimuma en yakın sıralama algoritması olduğunu söyledi. Teorik minimumu $n \log n$ diye söylesek de pratikte biraz daha fazla adım yapıldığını, ama teoriye en yakın algoritmanın merge sort olduğunu ekledi.

> **Ek bilgi (derste anlatılmadı):** Karşılaştırmaya dayalı her sıralama algoritması en kötü durumda en az $\lceil \log_2 n! \rceil$ karşılaştırma yapar. Bu değer yaklaşık $n\log_2 n - 1{,}44n$'dir. Merge sort'un en kötü durumu olan $n\log_2 n - n + 1$ bu sınıra çok yakındır. Örneğin $n = 32$ için alt sınır 118, merge sort'un en kötü durumu 129'dur.

7. Quick Sort'un Çalışma Mantığı ve Bir Bölümleme İzi

7.1 Genel Fikir

Hoca quick sort'un temel mantığını sınıfa sordu. Bir öğrenci şöyle özetledi: Önce diziden bir eleman seçilir. Bu elemandan büyükler sağ tarafa, küçükler sol tarafa yerleştirilir. Sonra bölünen iki tarafta da yine birer eleman seçilir ve aynı işlem yapılır. Bu, tek eleman kalana kadar sürer. Hoca bunun genel konsepti doğru özetlediğini söyledi.

Burada bir pivot elemana ihtiyaç vardır. Küçükten büyüğe sıralıyorsak pivottan küçükler başa, büyükler sona gidecek şekilde düzenleme yapılır. Bunun için soldan ve sağdan birer indis başlatılır: i ve j. Soldan sağa ilerlerken hedef pivottan büyük bir eleman bulmaktır. Sağdan sola ilerlerken hedef pivottan küçük bir eleman bulmaktır. İkisi de bulununca yerleri değiştirilir.

7.2 Bölümleme İzi

Hocanın ekrandaki örneğinde pivot 5'ti. Altyazıda başlangıç dizisi okunmuyor. Hocanın anlattığı yer değiştirmeler ve verdiği son dizi geriye doğru izlenince başlangıç dizisi $5, 3, 1, 9, 8, 2, 4, 7$ çıkar (ara adım eklendi).

Adım	i durduğu eleman	j durduğu eleman	İşlem	Dizi
Başlangıç			pivot $p = 5$	$5, 3, 1, 9, 8, 2, 4, 7$
1	9	4	$9 \leftrightarrow 4$	$5, 3, 1, 4, 8, 2, 9, 7$
2	8	2	$8 \leftrightarrow 2$	$5, 3, 1, 4, 2, 8, 9, 7$
3	8	2	indisler kesişti, pivot $\leftrightarrow 2$	$2, 3, 1, 4, 5, 8, 9, 7$

Hoca son adımı şöyle anlattı: i ile j eşit olduğunda ya da birbirlerini geçtiklerinde 2 başa alınır, 2'nin yerine 5 konur. Sol tarafta $2, 3, 1, 4$, sağ tarafta $8, 9, 7$ kalır. Pivot 5 doğru yerine gitmiştir. Python ile bu iz doğrulandı ve bölümlemede 9 karşılaştırma sayıldı.

Hoca bu yaklaşımın avantajını şöyle anlattı: Bir turda en az iki elemanın yerini değiştirme şansı vardır. Küçük olan soldan başa, büyük olan baştan sona doğru tek bir yer değiştirmeyle taşınır. Bu sayede çok daha hızlı ilerleme kaydedilir.

8. Quick Sort ve Hoare Bölümlemesi Sözde Kodu

8.1 Quick Sort

Hoca merge sort ile farkı bir kez daha vurguladı: Quick sort'ta efor bölme sırasında harcanır, birleştirme görece basittir. Merge sort'ta ise ikiye bölerken hiçbir iş yapılmıyor, birleştirirken merge operasyonu yapılıyordu. Bu yüzden quick sort'ta $f(n)$ fonksiyonunu algoritmanın ilk kısmı, yani bölümleme belirler.

Koşul $l < r$ olduğu sürece özyineleme devam eder. Her adımda pivot olması gereken yere yerleştirilir. Bölme ortadan değil, pivotun yerleştiği s konumundan yapılır.

```
Quicksort(A[l..r])
  if l < r
    s <- HoarePartition(A[l..r])
    Quicksort(A[l..s-1])
    Quicksort(A[s+1..r])
```

8.2 Hoare Bölümlemesi

Hoca bölümleme (partitioning) için çeşitli algoritmalar olduğunu, bunların birbirine göre artı ve eksileri bulunduğunu söyledi. Derste Hoare'nin algoritması (Hoare partitioning) kullanıldı. $A[l..r]$ kısmı gönderilir. Pivot $p = A[l]$, yani dizinin ilk elemanı seçilir. Hoca, algoritmanın verimini (efficiency) artırmak için farklı pivot seçimleri uygulanabileceğini de söyledi.

Hoca sözde kodu boşluklu yazıp sınıfla tamamladı:

- **İlk `repeat until` koşulu:** Bir öğrenci "i'nin j'den büyük olması" dedi. Hoca bunun olmadığını söyledi. Soldan ilerlerken pivottan büyük bir sayı aranır. Bir öğrenci "$A[i] < p$ olduğu sürece i artar" dedi. Hoca, `until` kullanıldığı için tersinin yazılması gerektiğini söyledi: `until A[i] >= p`.
- **İkinci `repeat until` koşulu:** Bir öğrenci "$A[j] < p$" dedi. Hoca bunu `until A[j] <= p` olarak düzeltti.
- **Sonraki satır:** İki eleman da bulununca yer değiştirilir: `swap(A[i], A[j])`.
- **Dış döngünün koşulu:** i, j'yi geçmeyecek. $i \geq j$ olana kadar devam edilir: `until i >= j`. Hoca bunun `while` ile tersinden de yazılabileceğini söyledi.
- **Son adım:** Pivot yerine konur. Pivotun yeri yukarıda saklanmıştı: $A[l]$. En sonunda $A[l]$ ile $A[j]$ yer değiştirir.

```
HoarePartition(A[l..r])
  p <- A[l]
  i <- l; j <- r + 1
  repeat
    repeat i <- i + 1 until A[i] >= p
    repeat j <- j - 1 until A[j] <= p
    swap(A[i], A[j])
  until i >= j
  swap(A[i], A[j])
  swap(A[l], A[j])
  return j
```

Hoca bu algoritmanın hem elemanların yerini değiştirdiğini, hem de bölmenin hangi noktadan yapılacağını, yani i ile j'nin buluştuğu noktayı belirlediğini söyledi.

> **Ders sonrası not:** Döngüden sonraki `swap(A[i], A[j])` satırının işlevi derste söylenmedi. İç döngüler $i \geq j$ olduktan sonra da bir yer değiştirme yapar. Bu satır o son, hatalı yer değiştirmeyi geri alır. Python ile bu sözde kod $5, 3, 1, 9, 8, 2, 4, 7$ üzerinde çalıştırıldı. Sonuç $2, 3, 1, 4, 5, 8, 9, 7$ ve $s = 4$ çıktı. Ayrıca i dizinin sonunu geçebilir. Bunun için büyük olasılıkla kitaptaki gibi dizinin sonuna pivottan büyük bir gözcü (sentinel) eleman konur.

9. Quick Sort'un En İyi Durum Analizi

9.1 Bağıntının Yazılması

Hoca en iyi durum (best case) bağıntısını sınıfa sordu. Bir öğrenci en iyi durumda dizinin zaten sıralı olacağını, bütün elemanların üzerinden birer kez geçileceğini, bu yüzden n yazılacağını söyledi. Hoca önce bağıntının baş kısmının söylenmediğini hatırlattı ve kendisi yazdı: Problem ikiye bölündüğü ve iki taraf da çözüldüğü için baş kısım $2C_b(n/2)$'dir. Öğrenci açıklamasını tekrar ettiğinde hoca "bence böyle değil" dedi. $f(n) = n$ sonucu doğruydu, ama en iyi durum senaryosu doğru ifade edilmemişti.

> **Ders sonrası not:** Pivot ilk eleman seçildiğinde zaten sıralı bir dizi quick sort'un en iyi değil, en kötü durumudur. Her bölümlemede bir taraf boş kalır ve $\Theta(n^2)$ karşılaştırma yapılır. En iyi durum, pivotun her seferinde diziyi tam ortadan bölmesidir.

Soru: Orada n değil de $\Theta(n)$ yazılması gerekmez mi?

Cevap: Hoca hayır dedi. Burası $f(n)$ fonksiyonudur ve arkadaşınız yerine n yazılması gerektiğini söyledi. Hoca asıl sorunun neden n yazıldığı olduğunu söyleyip soruyu bütün sınıfa yöneltti.

9.2 Neden $f(n) = n$?

Bir öğrenci, pivot olarak ortadaki bir eleman alınırsa en iyi durumun çıkıp çıkmayacağını sordu. Hoca bunu bir örnekle açıkladı. Pivot 5 olsun. Soldan sağa ilerlerken amaç büyük sayıları öbür tarafa atmaktır. Sol tarafta $9, 8, 7, 6$, sağ tarafta $4, 3, 2, 1$ olduğunu düşünelim. Önce 1 ile 9, sonra 2 ile 8, sonra 3 ile 7, en son 4 ile 6 yer değiştirir. Bu değişimler yapılırken n adet karşılaştırma yapılır.

$5, 9, 8, 7, 6, 4, 3, 2, 1$ dizisiyle iz şöyledir (ara adım eklendi):

Adım	İşlem	Dizi
Başlangıç	pivot $p = 5$	$5, 9, 8, 7, 6, 4, 3, 2, 1$
1	$9 \leftrightarrow 1$	$5, 1, 8, 7, 6, 4, 3, 2, 9$
2	$8 \leftrightarrow 2$	$5, 1, 2, 7, 6, 4, 3, 8, 9$
3	$7 \leftrightarrow 3$	$5, 1, 2, 3, 6, 4, 7, 8, 9$
4	$6 \leftrightarrow 4$	$5, 1, 2, 3, 4, 6, 7, 8, 9$
5	kesişme, pivot $\leftrightarrow 4$	$4, 1, 2, 3, 5, 6, 7, 8, 9$

Pivot tam ortaya yerleşir ve iki taraf eşit büyüklükte kalır. Hoca, en iyi durum olmasaydı yani indisler ortada buluşamasaydı, dizinin soluna ya da sağına doğru daha fazla ilerlemek ve daha çok elemanla karşılaştırma yapmak gerekeceğini, bunun en kötü durumda görüleceğini söyledi. En iyi durumda iki taraftan da yer değiştirerek ilerlenir. Hem sağdan hem soldan karşılaştırma yapıldığı için $n/2 + n/2$, toplamda n karşılaştırma yapılır. Bu yüzden $f(n) = n$'dir.

> **Ders sonrası not:** Python ile sayıldığında 9 elemanlı bu örnekte 10 karşılaştırma, ilk örnekte (8 eleman) 9 karşılaştırma çıktı. Kitaptaki analize göre büyük olasılıkla indisler kesişirse $n+1$, aynı yerde buluşursa n karşılaştırma yapılır. Büyüme derecesi bakımından $f(n) \in \Theta(n)$ olduğu için sonuç değişmez.

9.3 Master Teoremi ile Çözüm

$$C_b(n) = 2C_b(n/2) + n, \quad n > 1, \qquad C_b(1) = 0$$

Burada $a = 2$, $b = 2$, $d = 1$'dir. $a = b^d$ olduğu için:

$$C_b(n) \in \Theta(n \log n)$$

Python ile $n = 2, 4, 8, 16, 32$ için bağıntı $2, 8, 24, 64, 160$ verdi. Bu değerler $n \log_2 n$ ile aynıdır.

Soru: a'nın neden 2 olduğunu tam anlayamadım.

Cevap: Sayı grubu pivot elemana göre ikiye bölünür. Buradaki örnek sayıları her turda eşit olmak zorunda değildir. Ama iki grup da çözülmeye devam edilir. a, çözülmeye devam edilen alt problem sayısıdır. İkili aramada ikiye bölünmesine rağmen bir grupla ilgilenilmediği için $a = 1$ yazılmıştı.

> **Hocanın vurgusu:** Master teoreminde a, kaç alt problemin çözülmeye devam edildiğidir. Parçalar eşit olmasa da quick sort'ta $a = 2$'dir.

10. Gelecek Haftanın Planı

Hoca quick sort'un en kötü durumunu bu hafta yapmayacağını söyledi. Gelecek hafta en kötü durum çözülecek ve ortalama durumun (average case) formülü yazılacak. Böylece böl ve yönetteki iki temel problemin analizi tamamlanacak. Ardından bir uygulama sorusu çözülecek ve sonraki hafta hash algoritmalarına, yani hashing konusuna geçilecek.

Soru: Master teoremini sadece böl ve yönette mi kullanıyoruz?

Cevap: Hoca "evet, sadece böl ve yönet rekürans bağıntılarında kullanabiliyoruz" dedi.

> **Ders sonrası not:** Teorem, $T(n) = aT(n/b) + f(n)$ biçimindeki her bağıntıya uygulanır. İkili arama gibi sabit oranda azaltma (decrease by a constant factor) algoritmaları da bu biçimdedir ($a = 1$, $b = 2$, $d = 0$, sonuç $\Theta(\log n)$). Hocanın kendisi de bu derste ikili aramayı $a = 1$ örneği olarak kullandı.
>
> **Ek bilgi (derste anlatılmadı):** Master teoreminin üç durumu şöyledir: $a < b^d$ ise $T(n) \in \Theta(n^d)$, $a = b^d$ ise $T(n) \in \Theta(n^d \log n)$, $a > b^d$ ise $T(n) \in \Theta(n^{\log_b a})$. Bu derste hem merge sort hem quick sort'un en iyi durumu ikinci duruma düştü.

Örnek Sorular

Soru 1 (derste sözlü, 00:10:29): Merge sort'un rekürans bağıntısını yazın. Temel işlem nedir, başlangıç koşulu nedir?

> **Çözüm:** Temel işlem Merge içindeki `B[i] <= C[j]` karşılaştırmasıdır. $C_w(n) = 2C_w(n/2) + C_{merge}(n)$, $n > 1$, $C_w(1) = 0$. Tek eleman kalınca karşılaştırma yapılmaz.

Soru 2 (derste sözlü, 00:18:42): İki sıralı diziyi birleştirirken en kötü durumda kaç karşılaştırma yapılır?

> **Çözüm:** $n - 1$. Örnek: $2, 4, 6, 8$ ile $1, 3, 5, 7$ birleştirilirken 1-2, 2-3, 3-4, 4-5, 5-6, 6-7, 7-8 karşılaştırılır. Toplam 7. En iyi durumda (bir yarının bütün elemanları diğerininkinden küçükse) $n/2$ karşılaştırma yapılır.

Soru 3 (derste sözlü, 00:21:48): Merge sort bağıntısını master teoremiyle çözerseniz a, b, d kaç olur ve sonuç ne çıkar?

> **Çözüm:** $a = 2$, $b = 2$, $f(n) = n - 1 \in \Theta(n)$ olduğu için $d = 1$. $a = b^d$ olduğundan $C_w(n) \in \Theta(n^d \log n) = \Theta(n \log n)$.

Soru 4 (derste sözlü, 00:23:54): $f(n)$'i nasıl bulduk, neden $n - 1$?

> **Çözüm:** En kötü durumda iki yarıdan elemanlar dönüşümlü gelir ve son eleman hariç her eleman bir karşılaştırmayla yerleşir. Dizilerden biri bitince kalan eleman karşılaştırmasız kopyalandığı için n değil $n - 1$ olur.

Soru 5 (derste sözlü, 00:36:30): Hoare bölümlemesinde ilk `repeat until` döngüsünün koşulu ne olmalıdır? Altındaki satırlara ne yazılmalıdır?

> **Çözüm:** `until A[i] >= p`, sonra `until A[j] <= p`, ardından `swap(A[i], A[j])`, dış döngü `until i >= j`. En sonda `swap(A[l], A[j])` ile pivot yerine konur.

Soru 6 (derste sözlü, 00:41:11): Quick sort'un en iyi durum bağıntısını yazın. $f(n)$ yerine neden n yazılır?

> **Çözüm:** $C_b(n) = 2C_b(n/2) + n$, $C_b(1) = 0$. En iyi durumda pivot diziyi tam ortadan böler. i ve j iki uçtan ilerleyip ortada buluşur. $n/2 + n/2 = n$ karşılaştırma yapılır. Master teoremi: $a = 2$, $b = 2$, $d = 1$, sonuç $\Theta(n \log n)$.

Soru 7 (derste sözlü, 00:46:11): Quick sort'ta parçalar eşit olmak zorunda değilken a neden 2'dir?

> **Çözüm:** a, çözülmeye devam edilen alt problem sayısıdır. Quick sort'ta pivotun iki tarafı da sıralanır. İkili aramada yalnız bir yarıyla devam edildiği için $a = 1$'dir.

Soru 8 (derste sözlü, 00:48:09): Master teoremi yalnız böl ve yönet bağıntılarında mı kullanılır?

> **Çözüm:** Hoca "evet" dedi. Ancak teorem $T(n) = aT(n/b) + f(n)$ biçimindeki her bağıntıya uygulanır. İkili arama gibi sabit oranda azaltma algoritmaları da bu biçimdedir (bkz. Bölüm 10'daki ders sonrası not).

Dersin Toparlaması

Bu derste böl ve yönetin iki temel sıralama algoritmasının analizi yapıldı. Merge sort'ta bölme iş gerektirmez, asıl iş birleştirmededir. Temel işlem karşılaştırma alınınca en kötü durum bağıntısı $C_w(n) = 2C_w(n/2) + n - 1$, $C_w(1) = 0$ olarak yazıldı. $n - 1$ terimi, iki sıralı yarının birleştirilmesinin en kötü durumundan gelir. Master teoremiyle $a = b^d$ durumu elde edildi ve sonuç $\Theta(n \log n)$ çıktı. Merge sort, karşılaştırma sayısında teorik minimuma en yakın sıralama algoritmasıdır.

Quick sort'ta durum terstir: Asıl iş bölümlemededir. Hoare bölümlemesi ilk elemanı pivot alır, iki uçtan ilerleyen i ve j indisleriyle yanlış taraftaki elemanları yer değiştirir ve pivotu j konumuna yerleştirir. Pivotun her seferinde diziyi tam ortadan böldüğü en iyi durumda $C_b(n) = 2C_b(n/2) + n$ olur ve sonuç yine $\Theta(n \log n)$'dir. Gelecek hafta quick sort'un en kötü ve ortalama durumu işlenecek, bir uygulama sorusu çözülecek ve ardından hashing konusuna geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.